

接线说明

动手之前 所有接线、拔插操作都必须在断电状态下进行。带电插拔极易烧毁传感器或主控板的 IO 口。接好线后先通电检查电压是否正确，再下载程序。

1.1 引脚总览表

下表是本系统用到的全部引脚。前三组是球杆系统必需的，后面几组是调试和辅助功能。

引脚	功能	所属外设	接到哪里 / 说明
PD2	串口接收	UART5_RX	接 STP23L 模块的 TX（必须交叉接）
PC12	串口发送	UART5_TX	接 STP23L 模块的 RX（本项目可不接）
PA1	模拟输入	ADC2_IN1	接角位移传感器的信号输出脚
PC6	STEP 脉冲	TIM8_CH1	接步进驱动器 PUL+ / STEP+
PC8	DIR 方向	GPIO 输出	接步进驱动器 DIR+
PB14	EN 使能	GPIO 输出	接步进驱动器 ENA+
PE0	用户按键	GPIO 输入	板载按键，内部上拉，按下为低电平
PA9 / PA10	调试串口	USART1	接 USB 转 TTL，波特率 115200，看 printf 输出
PE8	指示灯	GPIO 输出	板载 LED
PA8	蜂鸣器	GPIO 输出	板载蜂鸣器
PD11~PD14	OLED 显示	GPIO 模拟时序	板载 OLED（DC / RES / SDA / SCL）
PB10 / PB11	IIC	软件 IIC	板载 ICM20948 陀螺仪（球杆系统不使用）
PB0	电池电压	ADC1_IN8	板载分压电路，用于电压监测

1.2 STP23L 激光测距模块

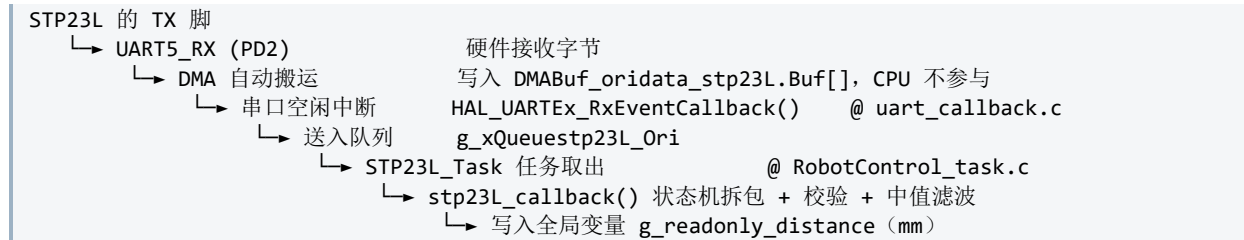
STP23L 通过串口不停地主动往外发数据，我们只负责接收，不需要给它发任何命令。因此实际上只有 RX 这一根信号线是必须接的。

STP23L 引脚	接到主控板	说明
-----------	-------	----

STP23L 引脚	接到主控板	说明
VCC	5V	先确认模块的供电电压要求，不要接错 3.3V / 5V
GND	GND	必须共地，这是最容易被忽略、也最容易出问题的一根线
TX	PD2 (UART5_RX)	数据线。串口必须交叉接：模块的 TX 接主控的 RX
RX	PC12 (UART5_TX)	本项目不向模块发命令，此线可不接

通信参数：波特率 230400，8 位数据位，1 位停止位，无校验。这些参数在 Core/Src/usart.c 的 MX_UART5_Init() 中配置，不要随意改动。

数据接收采用「DMA + 串口空闲中断」方式，完整链路如下：



安装要点 激光头装在导轨的一端，正对着小球滚动的方向。要保证小球在整个行程内都能被激光打到，且中间没有遮挡。模块的量程和盲区要与导轨长度匹配 —— 代码里默认的有效范围是 1mm~300mm，如果你的导轨更长，需要同步修改 bsp_stp23L.c 与 RobotControl_task.c 中的范围宏。

1.3 角位移传感器

角位移传感器本质上是一个「可以旋转的电位器」，可以理解成一个能转动的分压电阻：两端分别接 3.3V 和 GND，中间抽头输出的电压随转角线性变化。管子转到不同角度，抽头电压就不同，主控用 ADC 把这个电压读成 0~4095 的数字。

传感器引脚	接到主控板	说明
VCC (一端)	3.3V	务必接 3.3V，不能接 5V
GND (另一端)	GND	共地
OUT (中间抽头)	PA1 (ADC2_IN1)	模拟信号输出，直接进 ADC

为什么必须接 3.3V 而不是 5V STM32 的 ADC 输入上限就是 3.3V。如果给传感器供 5V，转到极限位置时输出会超过 3.3V，轻则读数被削顶（一直是 4095），重则损坏引脚。

代码中「角度」这个量的单位不是「度」，而是直接用 ADC 计数值（0~4095）。这样做省掉了一次换算，也避免引入额外的标定误差——PID 参数本身就是针对这个单位整定的。

软件上对 ADC 做了 80 点滑动平均（见 bsp_adc.c），能有效削弱单次采样的随机跳动，读数更稳。

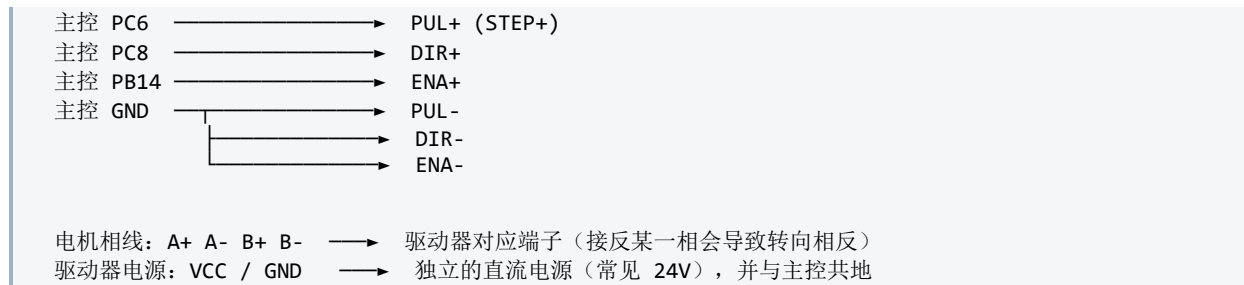
1.4 步进电机与驱动器

步进驱动器（TB6600、DM542、D36A 这一类）只认三组信号。理解了这三根线，步进电机的控制就没有秘密了：

信号	主控引脚	作用
STEP / PUL	PC6	每来一个脉冲，电机转一个「步距角」。脉冲频率越高，转速越快
DIR	PC8	高 / 低电平决定顺时针还是逆时针
ENA / EN	PB14	使能。失能时电机线圈断电，可以用手自由转动

共阴极接法（推荐）

大多数驱动器的信号端子是 PUL+/PUL-、DIR+/DIR-、ENA+/ENA- 三对。最常用的是共阴极接法：



共地是必须的 主控板的 GND 与步进驱动器电源的 GND 必须连在一起，否则脉冲信号没有共同的参考点，电机会出现完全不动、乱转或抖动等各种诡异现象。

驱动器拨码设置

驱动器侧面通常有一排拨码开关，用来设定细分数和输出电流。代码中的默认假设是：

项目	默认值	说明
电机步距角	1.8°	即转一圈需要 $360 \div 1.8 = 200$ 个整步
细分数	16	驱动器拨码设定，一个整步被细分成 16 小步
每圈脉冲数	3200	200×16 ，对应宏 ROLLBALL_STEPPER_PULSES_PER_REV
输出电流	按电机额定值设	设太大电机发烫，设太小力矩不够会丢步

重要：如果你改动了驱动器的细分拨码，必须同步修改
WHEELTEC_BSP/Inc/bsp_rollball_stepper.h 中的 ROLLBALL_STEPPER_MICROSTEP_DIVISION
宏，否则转速换算会全部错位。

EN 使能电平

不同品牌驱动器的 EN 引脚有效电平不一样，有的高电平使能，有的低电平使能。代码里用一个宏统一控制：

```
// 文件: WHEELTEC_BSP/Inc/bsp_rollball_stepper.h
#define ROLLBALL_STEPPER_ENABLE_ACTIVE_LEVEL GPIO_PIN_SET      // 高电平使能

// 若你的驱动器是低电平使能，改成：
// #define ROLLBALL_STEPPER_ENABLE_ACTIVE_LEVEL GPIO_PIN_RESET
```

怎么判断设反了 现象一：上电后电机一直发烫、被牢牢锁死，无论按不按启动键都松不开。现象二：长按启动键后电机仍能被手轻松转动、完全不响应。出现任一现象，都把这个宏改成相反的电平试试。

代码阅读指南

2.1 工程目录结构

本工程是在 WHEELTEC 小车底盘工程的基础上改造而来的，因此目录里保留了大量与球杆系统无关的文件。先看清楚哪些要读、哪些可以直接跳过：

目录	是否要读	内容
WHEELTEC_APP/	★ 重点	应用层任务：控制任务、显示任务、串口回调等
WHEELTEC_BSP/	★ 重点	板级驱动：测距、ADC、步进电机、按键、OLED 等
Core/Src/	部分要读	CubeMX 生成的外设初始化：时钟、GPIO、串口、定时器
Drivers/	不用读	ST 官方 HAL 库，不要修改
Middlewares/	不用读	FreeRTOS 与 USB 主机协议栈源码
USB_HOST/	不用读	USB 手柄相关，球杆系统不使用
MDK-ARM/	—	Keil 工程文件，双击 WHEELTEC.uvprojx 打开工程

2.2 建议阅读顺序

按下面的顺序读，是一条从整体到细节的路径。每一步读完都应该能回答对应的问题。

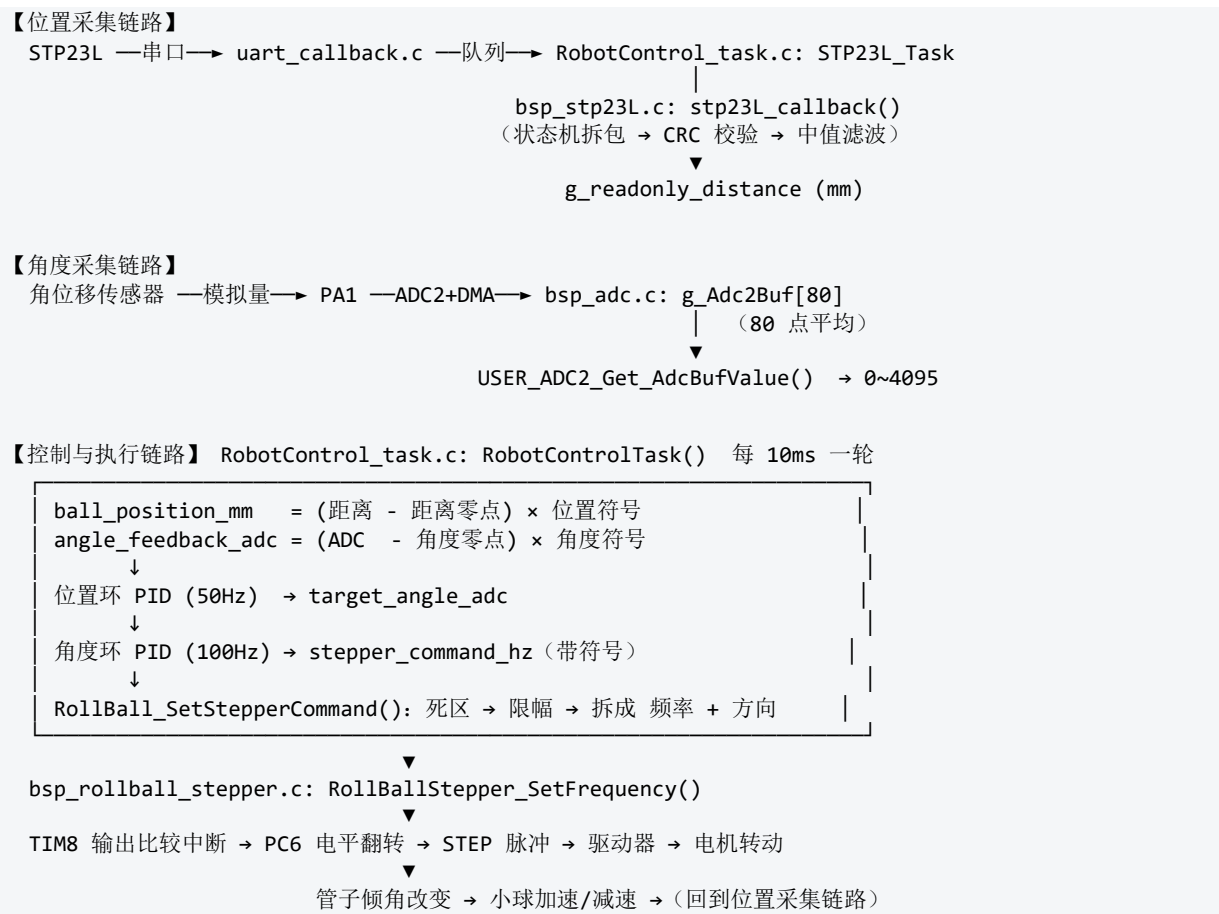
顺序	文件	读完应该能回答
1	RobotControl_task.c（开头注释 + 配置宏区）	系统整体是怎么运转的？有哪些可调参数？
2	同上：RobotControlTask() 主循环	每 10ms 到底做了哪些事？四种模式各是什么？
3	同上：RollBall_PID_Update()	PID 三项是怎么算的？为什么要滤波和限幅？
4	bsp_stp23L.c	激光测距的数据是怎么从串口变成一个 mm 数值的？
5	bsp_rollball_stepper.c	脉冲是怎么发出来的？换向为什么要先停？
6	bsp_adc.c	角度反馈是怎么采集和平均的？
7	Core/Src/freertos.c	系统里一共有哪些任务？各自的优先级是多少？
8	WHEELTEC_APP/show_task.c	OLED 上显示的那几个数字是从哪来的？

跳过技巧 RobotControl_task.c 里 RobotControlTask() 之后的所有函数

(RobotControlTask_NoUse、Set_Pwm、Read_Encoder、Drive_Motor、运动学正反解、IMU 走直线等) 全部是原小车底盘的遗留代码，球杆系统完全用不到，可以整段跳过。唯一还在被使用的是文件末尾的 target_limit_float()。

2.3 数据流全景图

这张图把所有文件串成了一条线。读代码时如果迷路了，回来看这张图：



2.4 核心文件逐个说明

RobotControl_task.c —— 系统的大脑

最重要的一个文件。文件开头有一大段总览注释，配置宏区集中在前 130 行左右。

函数	作用
RobotControlTask()	主控制任务，100Hz 固定周期。读传感器 → 按键 → 串级 PID

函数	作用
	→ 下发指令
RollBall_PID_Update()	PID 计算核心。位置环和角度环共用这一个函数，靠传入不同的控制器结构体区分
RollBall_PID_Reset()	清空 PID 历史状态。启停切换、修改目标值时必须调用
RollBall_SetStepperCommand()	把带符号的频率指令翻译成「方向电平 + 无符号频率」，并处理死区与限幅
RollBall_LimitFloat()	限幅工具函数，把一个数夹在上下限之间
STP23L_Task()	独立任务：从队列取原始数据，调用解析函数

bsp_stp23L.c —— 激光测距的翻译官

负责把串口收到的杂乱字节流拼成一帧完整数据，再滤波成一个干净的距离值。

- 状态机拆包：像找门牌号一样，先找到帧头（连续 4 个 0xAA），再按协议依次数出每个字段的字节数。
- CRC 校验：把帧头之后的所有字节累加，和帧尾的校验字节比对。不一致就整帧丢弃。
- 中值滤波：一帧里有 12 个测距点，先剔除超出 1 ~ 300mm 范围的异常点，剩下的取中值。

为什么用中值而不是平均值？平均值怕个别极端值 —— 12 个点里只要有 1 个因为反光突然变成 5mm，平均值就会被明显拉偏；而中值只看排序后中间那个，单个毛刺完全不影响。对于小球位置这种不能抖的信号，中值滤波效果好得多。

另外，代码刻意没有做跨帧的滑动平均。因为平均会引入延迟，而球杆系统是不稳定系统，传感器延迟会直接让闭环变得难以稳定。

bsp_rollball_stepper.c —— 脉冲发生器

对外只有几个函数，但内部有不少细节值得注意：

函数	作用
RollBallStepper_Init()	配置 DIR / EN 引脚，把 TIM8 摆到「停止且不输出」的安全状态
RollBallStepper_SetEnabled()	使能 / 失能驱动器。失能前会自动先停脉冲
RollBallStepper_SetFrequency()	设置频率和方向，控制电机的唯一入口

函数	作用
RollBallStepper_Stop()	停止脉冲输出（EN 状态不变，电机仍被锁住）
RollBallStepper_TIM8_CC_IRQHandler() ()	中断处理：把 CCR1 往后挪一个间隔，安排下一次翻转

三个关键设计：

- 同方向改频率不重启：只更新间隔值，中断下次自然按新间隔安排，脉冲流完全不中断，电机不会有顿挫。只有首次启动或需要换向时才重启定时器。
- 换向必须先停：步进电机在有脉冲的情况下改 DIR 电平会丢步甚至堵转。
- 临界区保护：修改状态时临时关中断，并且保存 / 还原 PRIMASK，避免破坏调用者本身的中断保护。

高级定时器的坑 TIM1 和 TIM8 是高级定时器，比普通定时器多一个主输出总开关 MOE。忘记 SET_BIT(TIM8->BDTR, TIM_BDTR_MOE) 的话，寄存器配置全对但引脚就是不出波形 —— 这是一个非常经典、非常难查的错误。

bsp_adc.c —— 角度采集

上电调用一次 ADC_Userconfig_Init() 之后，ADC 就在硬件层面连续转换，DMA 自动把结果写进数组，写满自动从头再来（循环模式），CPU 全程不参与。任何时候想要数据，直接调 Get 函数取数组平均值即可。

2.5 关键全局变量速查

变量	定义位置	含义
g_readonly_distance	bsp_stp23L.c	滤波后的激光测距值，单位 mm
g_rollball_debug	RobotControl_task.c	调试数据快照，OLED 和 Watch 窗口都从这里取数
g_rollball_control_enabled	RobotControl_task.c	闭环总开关：0 = 停止，1 = 运行中
g_xQueuestp23L_Ori	freertos.c	STP23L 原始数据队列，中断放、任务取

g_rollball_debug 结构体内部字段：

字段	含义
angle_adc_raw	角位移传感器原始读数（0~4095），标定角度零点时看这个
ball_distance_raw_mm	激光测得的原始距离（mm），标定位置零点时看这个
angle_feedback_adc	去掉零点、乘过符号后的角度：0 = 水平
ball_position_mm	去掉零点、乘过符号后的位置：0 = 导轨中点
target_position_mm	小球目标位置（mm），按键可改
ball_velocity_mm_s	小球速度（mm/s），稳定后应在 0 附近小幅波动
target_angle_adc	位置环算出的目标角度（模式 1 下借用作方向测试量）
stepper_command_hz	实际下发的脉冲频率，负号表示反方向
stepper_enabled	步进驱动器是否已使能
sensor_valid	本轮测距是否有效（0 = 被挡住或超量程，正在沿用旧值）

最好用的调试方法 在 Keil 中打开 View → Watch Windows → Watch 1，在空行输入 g_rollball_debug 回车，展开就能看到上面所有字段。全速运行时记得勾选 View → Periodic Window Update，数值才会实时刷新。这比串口打印直观得多，也不会拖慢控制周期。

2.6 FreeRTOS 任务一览

任务在 Core/Src/freertos.c 的 MX_FREERTOS_Init() 中创建。优先级数字越大越优先。

任务名	入口函数	优先级	作用
ControlTask	RobotControlTask	AboveNormal 1	★ 球杆系统主控制任务，100Hz
STP23L_Task	STP23L_Task	Normal	★ 激光测距数据解析
ImuTask	ImuTask	Normal	IMU 数据读取（球杆系统不使用）
BTControlTask	BlueToothControlTask	Normal	蓝牙 APP 控制解析
DataCheck	DataCheckTask	Normal	全局数据监测
showTask	show_task	BelowNormal 7	OLED 显示刷新
InitTask	StartDefaultTask	AboveNormal	开机初始化，执行完自删除

为什么控制任务优先级最高 控制任务对时间精度要求最高 —— 它靠 `vTaskDelayUntil` 维持严格的 10ms 周期，而 PID 里的 `dt` 是写死的常数。一旦周期被别的任务拖慢，微分和积分算出来的值就全错了。所以它必须能随时抢占显示、蓝牙这些不紧急的任务。

调试与标定流程参考

最重要的一条 务必按 模式 0 → 1 → 2 → 3 的顺序逐级调试，每一步确认没问题再进下一步。千万不要一上来就跑模式 3，否则电机可能带着机构撞到限位，损坏硬件。

3.1 四种调试模式

模式由 RobotControl_task.c 中的一个宏切换，改完需要重新编译下载：

```
// 文件: WHEELTEC_APP/RobotControl_task.c
#define ROLLBALL_CONTROL_MODE ROLLBALL_MODE_CASCADE_PID
```

模式	宏名	行为
0	ROLLBALL_MODE_SENSOR_MONITOR	电机完全不动，只打印传感器数据。最安全，用于标定零点
1	ROLLBALL_MODE_STEPPER_DIRECTION_TEST	按键点动电机 250ms，用于确认转向和接线
2	ROLLBALL_MODE_ANGLE_PID	只跑角度内环，目标固定为水平。用于整定内环参数
3	ROLLBALL_MODE_CASCADE_PID	完整串级控制，正式运行模式

3.2 按键操作说明

用户按键是板载的 PE0。三种操作被区分开来：

操作	模式 2 / 3	模式 1
长按 (>500ms)	启动 / 停止闭环（相当于急停）	启动 / 停止
单击	目标位置 +50mm	正向点动 200Hz
双击	目标位置 -50mm	反向点动 200Hz

状态切换后必须先松手才能触发下一次动作，这是为了防止一次长按被识别成两次而出现「开了又关」。

3.3 第一步：标定两个零点（模式 0）

这一步是所有后续调试的基础。用别人的零点值几乎一定跑不起来，必须自己实测。

1. 把模式宏改为 `ROLLBALL_MODE_SENSOR_MONITOR`，编译下载。
2. 用 USB 转 TTL 连到 PA9 / PA10，串口助手波特率设 115200，应能看到每 200ms 一行输出。
3. 用手把管子摆到水平，记下打印中 `adc=` 后面的数值，填入 `ROLLBALL_ANGLE_ADC_ZERO`。
4. 把小球放在导轨正中间，记下 `dist=` 后面的数值，填入 `ROLLBALL_BALL_DISTANCE_ZERO_MM`。
5. 改完重新下载，此时 `adc_d` 和 `pos` 两项都应该接近 0。

模式 0 的输出格式：

[RB][MON]	adc=2470	adc_d=0	dist=160.0mm	pos=0.0mm	valid=1	
						测距是否有效（0=被挡住或超量程）
						换算后的位置（标定后应为 0）
						原始距离，填入 <code>BALL_DISTANCE_ZERO_MM</code>
						与零点的差（标定后应为 0）
						原始 ADC，填入 <code>ANGLE_ADC_ZERO</code>

顺便在这一步确认三个符号方向是否自洽（这三条必须同时成立）：

- 手推小球往「位置正方向」走，`pos` 数值必须变大。否则改 `ROLLBALL_POSITION_SENSOR_SIGN` 的正负号。
- 手把管子往「能让球朝正方向滚」的方向掰，`angle` 数值必须变大。否则改 `ROLLBALL_ANGLE_SENSOR_SIGN`。
- 第三条在模式 1 里验证：步进电机发正频率，`angle` 必须变大。

符号搞反的后果 只要有一项符号反了，PID 就会变成「越纠越偏」——小球不但不会稳住，反而会加速冲向一端。这是新手最常见的失败原因。

3.4 第二步：确认电机方向（模式 1）

6. 模式宏改为 `ROLLBALL_MODE_STEPPER_DIRECTION_TEST`，编译下载。
7. 长按按键启动（此时驱动器被使能，电机会锁住）。
8. 单击按键，电机以 200Hz 正向转 250ms。
9. 观察串口打印中 `step=` 后面的数值——它是相对于点动前的 ADC 变化量。

观察结果	处理
step 变正	方向配置正确，无需改动

观察结果	处理
step 变负	把 ROLLBALL_STEPPER_POSITIVE_DIR_LEVEL 改成 GPIO_PIN_RESET
step 一直是 0	电机没转。检查 EN 电平宏、接线、驱动器电源、共地

模式 1 的输出格式：

[RB][STEP-DIR] en=1 cmd=200Hz dir=1 adc=2530 angle=-60.0 step=60

└ 看这个的正负

3.5 第三步：整定角度内环（模式 2）

改为 ROLLBALL_MODE_ANGLE_PID。此时目标角度固定为 0（水平），位置环不参与。长按启动后，用手轻轻推一下管子，观察它的反应，据此调整 ROLLBALL_ANGLE_KP：

现象	调整方向
回正很慢，软绵绵的	增大 Kp
回正后来回摆动、抖动、发出啸叫	减小 Kp
快速回正且停得干脆	合适，记下这个值

内环通常纯比例（P）就够了，不建议加 D —— ADC 噪声会被微分放大，反而让电机抖动。当前默认值 Kp = 8.0，含义是：1 个 ADC 的角度偏差对应 8Hz 的脉冲输出。

3.6 第四步：整定位置外环（模式 3）

改为 ROLLBALL_MODE_CASCADE_PID，这是正式运行模式。调参思路：

- 10.先把 ROLLBALL_POSITION_KD 设为 0，只留 Kp。此时小球一定会在目标位置附近来回震荡 —— 这是正常的，说明系统缺阻尼。
- 11.慢慢增大 Kd，震荡幅度会逐渐减小。找到「震荡刚好消失」的那个值。
- 12.如果小球反应太迟钝、迟迟推不回目标位置，适当增大 Kp，然后重新微调 Kd。
- 13.Ki 建议保持 0。球杆系统本身不稳定，积分项会让它更难稳住。

参数	默认值	作用
----	-----	----

常见问题排查

现象	可能原因	怎么查 / 怎么改
串口一直没有输出	串口线接错或波特率不对	确认接 PA9 / PA10, 波特率 115200, 确认共地
dist 一直是 0, valid=0	STP23L 没接好或接反	串口必须交叉: 模块 TX 接主控 PD2; 确认共地和供电
dist 数值乱跳	小球反光差、有遮挡、超量程	检查安装对准; 必要时调整 STP23L_DISTANCE_MAX_MM
adc 读数一直是 0 或 4095	角位移传感器供电或接线问题	确认接 3.3V 不是 5V; 确认信号线进的是 PA1
电机完全不动	EN 电平宏设反 / 没共地 / 驱动器没供电	先改 ROLLBALL_STEPPER_ENABLE_ACTIVE_LEVEL 试试
电机一直发烫且锁死	EN 电平宏设反	同上, 改成相反的电平
电机转但方向反了	DIR 宏或电机相线接反	改 ROLLBALL_STEPPER_POSITIVE_DIR_LEVEL
小球加速冲向一端	某个符号宏方向反了	回到模式 0, 逐条核对 4.3 节的三个符号方向
小球在目标附近来回震荡	阻尼不足	增大 ROLLBALL_POSITION_KD
管子高频抖动、有啸叫	内环 Kp 过大	减小 ROLLBALL_ANGLE_KP
一按启动电机就猛窜一下	PID 没被正确复位	检查启动分支里的 RollBall_PID_Reset 调用
OLED 无显示	OLED 排线接触不良	检查 PD11~PD14

附：修改代码时最常改动的几处

要改什么	改哪个宏	在哪个文件
------	------	-------

要改什么	改哪个宏	在哪个文件
切换调试模式	ROLLBALL_CONTROL_MODE	RobotControl_task.c
角度零点	ROLLBALL_ANGLE_ADC_ZERO	RobotControl_task.c
位置零点	ROLLBALL_BALL_DISTANCE_ZERO_MM	RobotControl_task.c
传感器方向	..._SENSOR_SIGN	RobotControl_task.c
电机方向	..._POSITIVE_DIR_LEVEL	RobotControl_task.c
PID 参数	ROLLBALL_POSITION_K* / ANGLE_K*	RobotControl_task.c
驱动器使能电平	..._ENABLE_ACTIVE_LEVEL	bsp_rollball_stepper.h
驱动器细分数	..._MICROSTEP_DIVISION	bsp_rollball_stepper.h
测距有效范围	STP23L_DISTANCE_MAX_MM	bsp_stp23L.c

小结 球杆系统的调试，九成的时间花在「标定零点」和「确认三个符号方向」这两件事上。这两件事做对了，PID 参数即使不完美，小球也能大致稳住；这两件事做错了，参数怎么调都不可能稳。